

Le basi

Sintassi di Base

Le regole fondamentali della sintassi del C++.

Le regole di base del C++

Ogni linguaggio ha le sue regole grammaticali. In italiano mettiamo il punto alla fine della frase. In C++ dobbiamo mettere il **punto e virgola** alla fine di ogni istruzione. Se non rispetti queste regole, il programma non funziona.

Imparare la sintassi di base è come imparare l'alfabeto: prima lo conosci, poi puoi scrivere qualsiasi cosa.

La regola più importante: il punto e virgola

Ogni istruzione in C++ deve terminare con un **punto e virgola** (;). Pensa a esso come al punto fermo di una frase.

```
int x = 10;           // istruzione 1 – termina con ;  
cout << "Ciao!";     // istruzione 2 – termina con ;  
x = x + 1;           // istruzione 3 – termina con ;
```

Dimenticare il punto e virgola è l'errore più comune per chi inizia. Il compilatore te lo segnala subito.

Le parentesi graffe: i blocchi

Le parentesi graffe { e } delimitano un **blocco di codice**, cioè un gruppo di istruzioni che stanno insieme.

Pensa a un blocco come a un paragrafo in un testo: delimita un insieme di cose che vanno lette insieme.

```
int main() {  
    // tutto qui dentro è il "blocco" del main  
    int x = 5;  
    cout << x;  
}
```

Ogni `{` apre un blocco, ogni `}` lo chiude. Devono sempre essere in coppia.

Maiuscole e minuscole: fanno differenza

Il C++ distingue tra lettere maiuscole e minuscole. `cout`, `Cout` e `COUT` sono tre cose completamente diverse (solo `cout` è quello giusto).

```
int eta = 16;  
int Eta = 20;    // questa è un'altra variabile, diversa dalla prima!  
  
cout << eta;     // stampa 16  
cout << Eta;     // stampa 20
```

Regola pratica: scrivi tutto in minuscolo, tranne dove le convenzioni prevedono il contrario.

L'indentazione: non obbligatoria, ma fondamentale

In C++, gli spazi a inizio riga (l'indentazione) non cambiano il comportamento del programma. Questo funziona:

```
int main(){int x=5;cout<<x;return 0;}
```

Ma è illeggibile. Scrivi sempre il codice così:

```
int main() {  
    int x = 5;  
    if (x > 0) {  
        cout << "positivo";  
    }  
    return 0;  
}
```

Usa 4 spazi (o un tab) per ogni livello di indentazione. Il tuo futuro io te ne sarà grato.

I nomi delle variabili (identificatori)

Un **identificatore** è il nome che dai a una variabile, a una funzione, ecc. Pensa a esso come a un'etichetta su una scatola.

Regole per i nomi:

- Deve iniziare con una lettera (`a-z` , `A-Z`) o un underscore `_`
- Può contenere lettere, numeri e underscore
- Non può contenere spazi o simboli speciali (`@` , `!` , `-` , ecc.)
- Non può essere una parola riservata del linguaggio (come `int` , `if` , `return`)

```
// Nomi validi  
int eta;  
double altezza_media;  
string nomeUtente;  
int x1;  
  
// Nomi non validi  
int 2variabile;    // inizia con un numero - ERRORE  
int nome utente;  // contiene uno spazio - ERRORE  
int int;           // è una parola riservata - ERRORE
```

Le parole riservate

Le **parole riservate** sono parole che hanno già un significato speciale in C++. Non puoi usarle come nomi di variabili:

```
int, float, double, char, bool, void  
if, else, while, for, do, switch, case, break, continue  
return, class, struct, public, private, new, delete
```

Le convenzioni di stile

In C++ esistono tre modi principali per scrivere i nomi:

camelCase — la prima parola è minuscola, le successive iniziano con la maiuscola. Usato per le variabili:

```
int numeroStudenti;  
string nomeUtente;
```

snake_case — tutto minuscolo, parole separate da underscore. Anch'esso usato per le variabili:

```
int numero_studenti;  
string nome_utente;
```

PascalCase — ogni parola inizia con la maiuscola. Usato per i nomi delle classi:

```
class NomeClasse;  
struct DatiStudente;
```

Scegli uno stile e usalo in modo coerente in tutto il progetto.

I tipi di istruzione

In C++ ci sono diversi tipi di istruzioni. Ecco le principali:

```
// Dichiarazione di variabile – riserva uno spazio in memoria
int x;

// Assegnazione – mette un valore nella variabile
x = 10;

// Dichiarazione con inizializzazione – le due cose insieme
int y = 20;

// Chiamata a funzione – esegue un'azione
cout << "Ciao";

// Istruzione condizionale – prende una decisione
if (x > 0) {
    cout << "positivo";
}

// Istruzione di ritorno – termina la funzione
return 0;
```

Esempio completo

```
#include <iostream>
using namespace std;

int main() {
    // Dichiariamo e inizializziamo tre variabili
    int numero = 42;
    double pi = 3.14159;
    string messaggio = "Il numero magico è: ";

    // Stampiamo il messaggio e il valore di numero
    cout << messaggio << numero << endl;

    // Stampiamo il valore di pi
    cout << "Pi greco: " << pi << endl;

    // Calcoliamo il doppio di numero e lo stampiamo
    int doppio = numero * 2;
    cout << "Il doppio è: " << doppio << endl;

    return 0;
}
```

Output:

```
Il numero magico è: 42
Pi greco: 3.14159
Il doppio è: 84
```

Commenti in C++

Come usare i commenti in C++ per documentare il codice.

A cosa servono i commenti?

Immagina di tornare a leggere un codice che hai scritto sei mesi fa. Senza commenti, potresti non capire più cosa fa o perché lo hai scritto in quel modo.

I **commenti** sono note che scrivi nel codice per spiegare cosa sta succedendo. Il compilatore le ignora completamente: esistono solo per chi legge il codice (tu, un compagno, un professore).

Scrivere buoni commenti è un'abitudine che distingue un programmatore attento da uno frettoloso.

Commento su riga singola: //

Il commento su riga singola inizia con `//`. Tutto ciò che viene dopo `//`, fino alla fine della riga, viene ignorato dal compilatore.

```
// Questo intero commento viene ignorato
int x = 5; // anche questo, solo la parte dopo //

// Puoi usare i commenti per disabilitare temporaneamente una riga
// int y = 10; ← questa riga non viene eseguita
```

Commento su più righe: /* . . . */

Il commento su più righe inizia con `/*` e finisce con `*/`. Tutto ciò che sta in mezzo viene ignorato, anche se occupa più righe.

```
/* Questo è un commento
   che occupa più righe.
   Utile per spiegazioni lunghe. */

int x = 5;

/*
 * Alcuni programmatori aggiungono un asterisco
 * all'inizio di ogni riga per migliorare la leggibilità.
 * È solo una convenzione stilistica.
 */
```

Cosa commentare: il "perché", non il "cosa"

Un buon commento spiega **perché** fai qualcosa, non **cosa** stai facendo. Il codice stesso dovrebbe essere abbastanza chiaro da spiegare il "cosa".

```
// Commento inutile – il codice si capisce già da solo
x = x + 1; // incrementa x di 1

// Commento utile – spiega il motivo
tentativi = tentativi + 1; // blocchiamo l'accesso dopo 3 tentativi falliti
```

Documentare le funzioni

Usa i commenti per spiegare cosa fa una funzione, i valori che riceve e quello che restituisce:

```
// Calcola l'area di un rettangolo.
// base: la larghezza del rettangolo (in centimetri)
// altezza: l'altezza del rettangolo (in centimetri)
// Restituisce l'area come numero decimale.
double calcolaArea(double base, double altezza) {
    return base * altezza;
}
```

Chi usa questa funzione capisce subito come usarla, senza dover leggere il codice al suo interno.

Disabilitare codice temporaneamente

Durante il debug, puoi commentare del codice per escluderlo momentaneamente dall'esecuzione:

```
int main() {
    int x = 10;
    // cout << "Valore intermedio: " << x << endl; ← disabilitato per ora
    cout << "Valore finale: " << x << endl;
    return 0;
}
```

Questo è più veloce che cancellare il codice e poi riscriverlo.

Commenti da evitare

Evita i commenti che non aggiungono alcuna informazione rispetto al codice:

```
// Da evitare – sono ovvi  
int eta = 16;    // imposta eta a 16  
x = x + 1;      // aggiunge 1 a x  
return 0;       // ritorna 0
```

Evita anche i commenti sbagliati, cioè che non corrispondono più al codice. Sono peggio di nessun commento, perché ingannano chi legge:

```
// Somma due numeri ← SBAGLIATO! In realtà moltiplica  
int risultato = a * b;
```

Esempio completo

```
#include <iostream>
using namespace std;

/*
 * Programma: Calcolatore dell'area di un cerchio
 * Autore: Alice
 *
 * Legge il raggio dall'utente e calcola l'area.
 */

// Valore approssimato di pi greco – non cambia mai
const double PI = 3.14159265358979;

int main() {
    double raggio;

    // Chiediamo il raggio all'utente
    cout << "Inserisci il raggio: ";
    cin >> raggio;

    // Calcoliamo l'area con la formula:  $A = \pi * r^2$ 
    double area = PI * raggio * raggio;

    cout << "L'area del cerchio è: " << area << endl;

    return 0;
}
```

Costanti

Come definire valori costanti in C++ con const e

Cos'è una costante?

Immagina di scrivere un programma che usa il numero pi greco (3.14159...) in dieci punti diversi. Se un giorno vuoi usare una versione più precisa, devi cambiarla in dieci posti.

Oppure potresti avere il numero `18` sparso nel codice. Chi legge non sa se è l'età minima, un limite di velocità o qualcos'altro.

Le **costanti** risolvono entrambi i problemi: dai un nome a un valore che non cambia mai, e lo scrivi in un unico posto.

La parola chiave `const`

Scrivi `const` prima del tipo per rendere una variabile costante. Una volta assegnato il valore, non può più essere cambiato.

```
const double PI = 3.14159265358979;  
const int ORE_PER_GIORNO = 24;  
const int ETA_MINIMA = 18;
```

Se provi a modificare una costante, il compilatore ti blocca con un errore:

```
const int MAX = 100;  
MAX = 200; // ERRORE: non puoi modificare una costante
```

Questo è un vantaggio: il compilatore ti protegge da modifiche accidentali.

La convenzione dei nomi

Per le costanti si usa la convenzione di scrivere **tutto in maiuscolo**, con le parole separate da underscore. Così si riconoscono a colpo d'occhio nel codice:

```
const double PI = 3.14159;  
const int ETA_MINIMA = 18;  
const int MAX_TENTATIVI = 3;  
const double VELOCITA_LUCE = 299792458.0;
```

Un altro modo: `#define`

Prima che esistesse `const`, i programmatori usavano la direttiva `#define`:

```
#define PI 3.14159
#define MAX_STUDENTI 30
```

Con `#define`, il compilatore sostituisce ogni occorrenza di `PI` con `3.14159` prima di compilare il codice. Non è una vera variabile: è una semplice sostituzione di testo.

```
#include <iostream>
#define PI 3.14159
#define RAGGIO 5

using namespace std;

int main() {
    // Il compilatore sostituisce PI con 3.14159 e RAGGIO con 5
    double area = PI * RAGGIO * RAGGIO;
    cout << "Area: " << area << endl;
    return 0;
}
```

const o #define ? Usa const

Oggi si preferisce sempre `const`. Ecco perché:

Caratteristica	<code>const</code>	<code>#define</code>
Ha un tipo definito	Sì	No
Il compilatore la controlla	Sì	No
Visibile nel debugger	Sì	No
Rispetta i blocchi di codice	Sì	No

```
// Meglio: const ha tipo e viene controllata
const double PI = 3.14159;

// Meno sicuro: #define non ha tipo
#define PI 3.14159
```

constexpr : costanti calcolate durante la compilazione

In C++ moderno esiste anche **constexpr**. Si usa per costanti il cui valore può essere calcolato già durante la compilazione (prima ancora che il programma venga eseguito):

```
constexpr int SECONDI_PER_MINUTO = 60;
constexpr int MINUTI_PER_ORA = 60;
// Il compilatore calcola 60 * 60 = 3600 prima ancora di eseguire il
programma
constexpr int SECONDI_PER_ORA = SECONDI_PER_MINUTO * MINUTI_PER_ORA;
```

Per ora puoi usare tranquillamente **const**. Imparerai **constexpr** più avanti.

Esempio pratico

```
#include <iostream>
using namespace std;

// Costanti definite una volta sola, usate ovunque nel programma
const double PI = 3.14159265358979;
const double GRAVITA = 9.81; // accelerazione di gravità in m/s²
const int MAX_VOTO = 10;

int main() {
    // Calcolo dell'area di un cerchio con raggio 4
    double raggio = 4.0;
    double area = PI * raggio * raggio;
    cout << "Area cerchio (r=" << raggio << "): " << area << endl;

    // Calcolo dell'energia potenziale di un oggetto
    double massa = 70.0; // kg
    double altezza = 5.0; // m
    double energia = massa * GRAVITA * altezza;
    cout << "Energia potenziale: " << energia << " J" << endl;

    // Uso della costante del voto massimo
    int voto = 8;
    cout << "Voto: " << voto << "/" << MAX_VOTO << endl;

    return 0;
}
```

Output:

```
Area cerchio (r=4): 50.2655
Energia potenziale: 3434.5 J
Voto: 8/10
```

Tipi di Dati

I tipi di dati fondamentali del C++: int, float, double, char e bool.

Perché esistono i tipi di dati?

Immagina di avere dei cassetti in un armadio. Un cassetto è per i calzini, uno per le magliette, uno per i pantaloni. Non metteresti dei pantaloni nel cassetto dei calzini.

In C++ funziona allo stesso modo: ogni variabile ha un **tipo** che stabilisce che genere di valore può contenere. A differenza di Python, non puoi mettere qualsiasi cosa in qualsiasi variabile: devi dichiarare il tipo in anticipo.

Questo rende il codice più sicuro e veloce.

Numeri interi: `int`

`int` (abbreviazione di *integer*, intero) contiene numeri senza virgola, sia positivi che negativi:

```
int eta = 16;
int temperatura = -5;
int anno = 2024;
int zero = 0;
```

Un `int` occupa 4 byte di memoria e può contenere valori da circa -2 miliardi a +2 miliardi. Per la maggior parte dei programmi, è più che sufficiente.

Se hai bisogno di numeri molto grandi o molto piccoli, esistono varianti:

```
short int piccolo = 32000; // massimo circa 32.000
long long enorme = 9000000000000LL; // fino a 9 miliardi di miliardi
```

Numeri decimali: `float` e `double`

Per numeri con la virgola (come 3.14 o 1.75) hai due opzioni.

`float`

Occupi 4 byte. Preciso fino a circa 7 cifre. Metti la `f` alla fine del numero per indicare che è un float:

```
float temperatura = 36.6f;  
float pi = 3.14159f;
```

double

Occupi 8 byte. Preciso fino a circa 15-16 cifre. È quello più usato:

```
double altezza = 1.75;  
double pi = 3.14159265358979;
```

Quale usare?

Usa sempre **double**, a meno che tu non abbia un motivo specifico per usare **float**. Il **double** è più preciso e il tipo preferito in C++ per i numeri decimali:

```
// Possibile problema con float  
float a = 0.1f + 0.2f;  
cout << a; // potrebbe stampare 0.3000001 invece di 0.3  
  
// Meglio con double  
double b = 0.1 + 0.2;  
cout << b; // molto più preciso
```

Singolo carattere: char

char (abbreviazione di *character*, carattere) contiene un singolo carattere. I caratteri si scrivono tra **apici singoli** '...':

```
char lettera = 'A';  
char cifra = '7';  
char spazio = ' ';
```


Internamente, ogni carattere è memorizzato come un numero (il suo codice ASCII). Per esempio, `'A'` corrisponde al numero 65:

```
char c = 'A';
cout << c;      // stampa: A
cout << (int)c; // stampa: 65 (il codice numerico di 'A')
```

Vero o falso: `bool`

`bool` (abbreviazione di *boolean*, booleano) può contenere solo due valori: `true` (vero) o `false` (falso):

```
bool maggiorenne = true;
bool haPatente = false;
bool risultato = (5 > 3); // true, perché 5 è maggiore di 3
```

Quando stampi un `bool`, di default viene mostrato come `1` (true) o `0` (false). Per stampare le parole `true` e `false`:

```
bool x = true;
cout << x;      // stampa: 1
cout << boolalpha << x; // stampa: true
```

Quanto spazio occupano?

Puoi controllare quanti byte occupa un tipo con `sizeof`:

```
cout << sizeof(int) << endl; // 4
cout << sizeof(double) << endl; // 8
cout << sizeof(char) << endl; // 1
cout << sizeof(bool) << endl; // 1
```

Riepilogo

Tipo	Spazio	Valori
<code>int</code>	4 byte	numeri interi, da -2 miliardi a +2 miliardi
<code>float</code>	4 byte	decimali, ~7 cifre significative
<code>double</code>	8 byte	decimali, ~15 cifre significative
<code>char</code>	1 byte	un singolo carattere
<code>bool</code>	1 byte	solo <code>true</code> oppure <code>false</code>

Convertire tra tipi

A volte hai bisogno di convertire un valore da un tipo all'altro.

La **conversione implicita** avviene automaticamente quando è sicura:

```
int x = 5;
double y = x;    // int viene convertito in double automaticamente
cout << y;      // 5.0
```

La **conversione esplicita** (chiamata *casting*) si usa quando vuoi forzare una conversione:

```
double pi = 3.14159;
int parteIntera = (int)pi; // la parte decimale viene scartata
cout << parteIntera;      // 3 (non arrotondato, solo troncato)
```

Attenzione alla **divisione tra interi**: quando dividi due `int`, il risultato è sempre un intero (la parte decimale viene persa):

```
int x = 5, y = 2;
int risultato = x / y;    // risultato = 2, non 2.5!
double risultato2 = (double)x / y; // risultato2 = 2.5 (corretto)
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    // Una variabile per ogni tipo principale
    int eta = 16;
    double altezza = 1.75;
    char iniziale = 'A';
    bool maggiorenne = false;

    cout << "Età (int): " << eta << endl;
    cout << "Altezza (double): " << altezza << endl;
    cout << "Iniziale (char): " << iniziale << endl;
    cout << "Maggiorenne (bool): " << boolalpha << maggiorenne << endl;

    return 0;
}
```

Output:

```
Età (int): 16
Altezza (double): 1.75
Iniziale (char): A
Maggiorenne (bool): false
```

Installazione

Come installare il compilatore GCC e configurare Visual Studio Code per programmare in C++.

Cosa ti serve

Per scrivere programmi in C++ hai bisogno di due cose:

1. Un **compilatore**: il programma che trasforma il tuo codice C++ in qualcosa che il computer può eseguire
2. Un **editor di testo**: dove scrivi il codice

In questa guida usiamo **GCC** come compilatore e **Visual Studio Code** (VS Code) come editor. Entrambi sono gratuiti.

Differenza tra C++ e Python: la compilazione

In Python, scrivi il codice e lo esegui subito. In C++ c'è un passaggio intermedio: la **compilazione**.

È come la differenza tra un interprete simultaneo (Python — traduce mentre parli) e un traduttore che prima legge tutto il testo, poi ti consegna il testo tradotto (C++ — prima compila, poi esegui).

Il vantaggio è che i programmi C++ compilati sono molto più veloci di quelli Python.

Installare il compilatore GCC

GCC (GNU Compiler Collection) è il compilatore C++ più usato al mondo. È gratuito e open source.

Su Windows

Il modo più semplice è installare **MSYS2** che include GCC:

1. Vai su **msys2.org** e scarica il programma di installazione
2. Segui le istruzioni di installazione
3. Apri il terminale MSYS2 e digita: `pacman -S mingw-w64-ucrt-x86_64-gcc`
4. Aggiungi la cartella `C:\msys64\ucrt64\bin` alla variabile d'ambiente PATH

Verifica che funzioni aprendo il prompt dei comandi e digitando:

```
g++ --version
```

Se vedi un numero di versione, l'installazione è riuscita.

Su macOS

Su macOS puoi installare gli strumenti di sviluppo di Apple con questo comando nel terminale:

```
xcode-select --install
```

Una finestra ti chiederà conferma — clicca "Installa". Al termine, verifica:

```
g++ --version
```

Su Linux (Ubuntu/Debian)

Su Linux è il più semplice:

```
sudo apt update  
sudo apt install g++
```

Verifica:

```
g++ --version
```

Installare Visual Studio Code

VS Code è un editor di testo gratuito, leggero e molto popolare tra i programmatori.

1. Vai su **code.visualstudio.com** e scaricalo
2. Installalo come qualsiasi altro programma

Estensioni consigliate

Dopo aver aperto VS Code, installa queste estensioni (clicca sull'icona delle estensioni nella barra laterale sinistra):

- **C/C++** (di Microsoft): colora il codice, suggerisce completamenti, aiuta col debugging
- **Code Runner**: esegue il codice con un clic

Il primo programma: Hello, World!

Crea il file

1. Apri VS Code
2. Crea una nuova cartella per i tuoi progetti (es. `progetti-cpp`)
3. Apri quella cartella in VS Code (File > Apri cartella)
4. Crea un nuovo file chiamato `hello.cpp`

Scrivi il codice

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

Compila il programma

Apri il terminale integrato in VS Code (View > Terminal):

```
g++ hello.cpp -o hello
```

Questo comando dice al compilatore GCC di:

- leggere il file `hello.cpp`
- creare un eseguibile chiamato `hello`

Esegui il programma

```
# Su Linux/macOS
./hello

# Su Windows
hello.exe
```

Output:

```
Hello, World!
```

Il comando di compilazione in dettaglio

```
g++ hello.cpp -o hello -Wall -std=c++17
```

Parte	Significato
<code>g++</code>	Il compilatore
<code>hello.cpp</code>	Il file sorgente da compilare
<code>-o hello</code>	Il nome dell'eseguibile da creare
<code>-Wall</code>	Mostra tutti gli avvisi (consigliato!)
<code>-std=c++17</code>	Usa la versione C++17 del linguaggio

Non vuoi installare niente? Usa Replit

Se preferisci iniziare senza installare nulla, puoi usare **Replit** — un ambiente di sviluppo online che funziona direttamente nel browser. Vai su **replit.com**, crea un account gratuito, crea un nuovo progetto C++ e scrivi il codice lì.

Altre alternative:

- **Code::Blocks** — IDE specifico per C/C++, facile da usare
 - **Dev-C++** — semplice IDE per Windows
-

Operatori in C++

Gli operatori aritmetici, di confronto e logici in C++.

Cosa sono gli operatori?

Un programma senza operatori non può fare quasi nulla. Gli **operatori** sono i simboli che usano il programma per fare calcoli, confrontare valori e prendere decisioni.

Sono come i simboli matematici che già conosci (`+` , `-` , `*` , `/`), con qualche aggiunta specifica per la programmazione.

Operatori aritmetici

Questi operatori fanno calcoli matematici:

Operatore	Operazione	Esempio	Risultato
<code>+</code>	Addizione	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Sottrazione	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Moltiplicazione	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Divisione	<code>10 / 3</code>	<code>3</code> (intera!)
<code>%</code>	Modulo (resto)	<code>10 % 3</code>	<code>1</code>


```
int a = 10, b = 3;

cout << a + b << endl; // 13
cout << a - b << endl; // 7
cout << a * b << endl; // 30
cout << a / b << endl; // 3 - attenzione, non 3.33!
cout << a % b << endl; // 1 - il resto della divisione
```

La divisione intera

Quando si dividono due numeri interi, il risultato è sempre un intero. La parte decimale viene scartata, non arrotondata:

```
int x = 7, y = 2;
cout << x / y << endl; // 3 (non 3.5!)

// Per ottenere 3.5, converti uno dei due in double prima di dividere
double risultato = (double)x / y;
cout << risultato << endl; // 3.5
```

Il modulo %

Il modulo restituisce il **resto** della divisione intera. È utile per capire se un numero è pari o dispari:

```
cout << 10 % 3 << endl; // 1 (10 = 3×3 + 1)
cout << 9 % 3 << endl; // 0 (9 è divisibile per 3)
cout << 7 % 2 << endl; // 1 (7 è dispari)
cout << 8 % 2 << endl; // 0 (8 è pari)
```

Operatori di assegnazione

Oltre al semplice `=`, esistono operatori che modificano una variabile in un colpo solo:

Operatore	Equivalente	Esempio con $x = 10$
<code>=</code>	assegnazione semplice	<code>x = 5</code> → x vale 5
<code>+=</code>	<code>x = x + y</code>	<code>x += 3</code> → x vale 13
<code>-=</code>	<code>x = x - y</code>	<code>x -= 3</code> → x vale 7
<code>*=</code>	<code>x = x * y</code>	<code>x *= 2</code> → x vale 20
<code>/=</code>	<code>x = x / y</code>	<code>x /= 5</code> → x vale 2
<code>%=</code>	<code>x = x % y</code>	<code>x %= 3</code> → x vale 1

```
int x = 10;
x += 5;    // x è ora 15
x -= 3;    // x è ora 12
x *= 2;    // x è ora 24
x /= 4;    // x è ora 6
x %= 4;    // x è ora 2
```

Operatori di incremento e decremento

Aggiungere o sottrarre 1 è così comune in C++ che esistono operatori appositi:

```
int x = 5;
x++;    // x diventa 6 (aggiunge 1)
x--;    // x diventa 5 (sottrae 1)
```

Esistono anche nella forma con `++` prima della variabile:

```
int x = 5;
int b = x++; // prima assegna x a b (b = 5), poi incrementa x (x = 6)
int c = ++x; // prima incrementa x (x = 7), poi assegna a c (c = 7)
```

Per i principianti è sufficiente usare `x++` e `x--` da soli, senza usarli dentro espressioni complesse.

Operatori di confronto

Confrontano due valori e restituiscono `true` (vero) o `false` (falso):

Operatore	Significato	Esempio	Risultato
<code>==</code>	uguale a	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	diverso da	<code>5 != 3</code>	<code>true</code>
<code><</code>	minore di	<code>3 < 5</code>	<code>true</code>
<code>></code>	maggiore di	<code>5 > 3</code>	<code>true</code>
<code><=</code>	minore o uguale	<code>5 <= 5</code>	<code>true</code>
<code>>=</code>	maggiore o uguale	<code>3 >= 5</code>	<code>false</code>

```
int a = 5, b = 3;
```

```
cout << (a == b) << endl; // 0 (false - 5 non è uguale a 3)
cout << (a != b) << endl; // 1 (true - 5 è diverso da 3)
cout << (a > b) << endl; // 1 (true - 5 è maggiore di 3)
cout << (a < b) << endl; // 0 (false - 5 non è minore di 3)
```

Errore comune: confondere `==` (confronto) con `=` (assegnazione). Scrivere `if (x = 5)` invece di `if (x == 5)` cambia il valore di `x` invece di confrontarlo!

Operatori logici

Combinano più condizioni insieme:

Operatore	Significato	Esempio	Risultato
<code>&&</code>	E (AND)	<code>true && false</code>	<code>false</code>
<code> </code>	O (OR)	<code>true false</code>	<code>true</code>
<code>!</code>	NON (NOT)	<code>!true</code>	<code>false</code>

AND (&&): entrambe le condizioni devono essere vere

```
int eta = 20;
bool haPatente = true;

if (eta >= 18 && haPatente) {
    cout << "Puoi guidare." << endl;
}
// Entrambe le condizioni sono vere → stampa il messaggio
```

OR (||): basta che almeno una condizione sia vera

```
int ora = 14;

if (ora < 9 || ora > 18) {
    cout << "Il negozio è chiuso." << endl;
} else {
    cout << "Il negozio è aperto." << endl;
}
// ora = 14, nessuna delle due condizioni è vera → "aperto"
```

NOT (!): inverte il valore booleano

```
bool piove = false;

if (!piove) {
    cout << "Possiamo uscire!" << endl;
}
// !false è true → stampa il messaggio
```

La priorità degli operatori

Come in matematica, alcuni operatori hanno la precedenza su altri. Da quella più alta a quella più bassa:

1. `!` , `++` , `--`
2. `*` , `/` , `%`
3. `+` , `-`
4. `<` , `>` , `<=` , `>=`
5. `==` , `!=`
6. `&&`
7. `||`
8. `=` , `+=` , `-=` , ecc.

Quando hai dubbi, usa le parentesi per rendere esplicito l'ordine:

```
int risultato = 2 + 3 * 4;      // 14 - la moltiplicazione va prima
int risultato2 = (2 + 3) * 4;   // 20 - le parentesi cambiano tutto
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    int a = 15, b = 4;

    // Operatori aritmetici
    cout << "Somma: " << a + b << endl;      // 19
    cout << "Differenza: " << a - b << endl;  // 11
    cout << "Prodotto: " << a * b << endl;    // 60
    cout << "Quoziente: " << a / b << endl;   // 3 (divisione intera)
    cout << "Resto: " << a % b << endl;      // 3

    cout << endl;

    // Operatori di confronto
    cout << "a > b: " << (a > b) << endl;    // 1 (true)
    cout << "a == b: " << (a == b) << endl; // 0 (false)

    // Operatori logici
    bool cond = (a > 10) && (b < 10);
    cout << "a>10 E b<10: " << cond << endl; // 1 (true)

    return 0;
}
```

Struttura di un Programma C++

Le parti fondamentali che compongono ogni programma C++.

Come è fatto un programma C++?

Prima di scrivere il tuo primo programma, devi capire la sua struttura. È come imparare a costruire una casa: devi sapere dove vanno le fondamenta, i muri e il tetto, prima di poter arredare.

Ogni programma C++ segue una struttura fissa. Conoscerla ti permette di capire qualsiasi codice tu incontrerai.

La struttura minima

Ecco il programma C++ più semplice possibile:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!";
    return 0;
}
```

Questo programma stampa la parola "Ciao!" sullo schermo. Solo cinque righe, ma ognuna ha un ruolo preciso.

Parte 1: `#include <iostream>`

```
#include <iostream>
```

Questa riga dice al compilatore: "Ho bisogno della libreria `iostream`."

Pensa alle librerie come a delle cassette degli attrezzi: `iostream` è la cassetta che contiene gli strumenti per scrivere sullo schermo (`cout`) e leggere l'input dell'utente (`cin`).

Senza questa riga, il compilatore non saprebbe cosa significa `cout` e il programma non funzionerebbe.

Parte 2: `using namespace std;`

```
using namespace std;
```

Pensa a un **namespace** come a un cognome. Se in classe ci sono due ragazzi chiamati Marco, li distingui con il cognome: "Marco Rossi" e "Marco Bianchi".

`cout` e `cin` appartengono alla "famiglia" `std`. Scrivere `using namespace std;` dice al programma: "quando uso `cout`, intendo `std::cout`". Così puoi scrivere `cout` invece del nome completo

```
std::cout .
```

Parte 3: `int main() { ... }`

```
int main() {  
    // il programma inizia qui  
    return 0;  
}
```

La funzione `main()` è il **punto di partenza** di ogni programma C++. Quando avvii un programma, il computer cerca sempre `main()` e inizia a eseguire le istruzioni al suo interno.

- `int` — indica che `main` restituisce un numero intero al termine
- Le parentesi graffe `{` e `}` — delimitano il corpo della funzione
- `return 0;` — segnala che il programma è terminato senza errori (0 = tutto ok)

Parte 4: le istruzioni dentro `main()`

```
cout << "Ciao!";
```

Dentro `main()` scrivi le istruzioni del programma, una per riga. Ogni istruzione termina con il punto e virgola `;`.

`cout` stampa qualcosa sullo schermo. La freccia `<<` indica "manda questo testo a cout".

Un esempio più completo

Quando hai bisogno di più funzionalità, includi più librerie:


```

#include <iostream>
#include <string>
using namespace std;

int main() {
    // Dichiariamo due variabili
    string nome = "Alice";
    int eta = 16;

    // Le stampiamo sullo schermo
    cout << "Nome: " << nome << endl;
    cout << "Età: " << eta << endl;

    return 0;
}

```

Output:

```

Nome: Alice
Età: 16

```

Qui abbiamo aggiunto `#include <string>` perché vogliamo usare le stringhe di testo.

L'ordine delle sezioni

Un file C++ tipico segue sempre quest'ordine dall'alto verso il basso:

1. **Le librerie** — cosa includiamo
2. **Il namespace** — quale "famiglia" di nomi usare
3. **Variabili e costanti globali** — valori disponibili ovunque (opzionale)
4. **La funzione `main()`** — da dove parte il programma
5. **Altre funzioni** — pezzi di codice riutilizzabili (opzionale)

```
// 1. Librerie
#include <iostream>
#include <cmath>

// 2. Namespace
using namespace std;

// 3. Costante globale (opzionale)
const double PI = 3.14159;

// 4. Funzione main
int main() {
    double raggio = 5.0;
    // PI * r * r calcola l'area del cerchio
    double area = PI * raggio * raggio;
    cout << "Area: " << area << endl;
    return 0;
}
```

Le parentesi graffe delimitano i blocchi

Le parentesi graffe { } compaiono ovunque in C++: nelle funzioni, nei cicli, nelle condizioni. Ogni coppia apre e chiude un **blocco di codice**.

```
int main() {           // apre il blocco di main
    if (true) {        // apre un blocco if
        cout << "Ciao";
    }                  // chiude il blocco if
    return 0;
}                      // chiude il blocco di main
```

Ogni { deve avere il suo } corrispondente. Se ne manca uno, il compilatore segnala un errore.

Il punto e virgola alla fine di ogni istruzione

In C++, quasi tutte le istruzioni terminano con ; . Dimenticarlo è uno degli errori più comuni:

```
// Corretto
cout << "Ciao!" << endl;
int x = 5;

// Errato – manca il ;
cout << "Ciao!" << endl // il compilatore segnala un errore
int x = 5                // anche qui
```

Se non capisci un errore del compilatore, prima di tutto controlla che non manchi un punto e virgola.

Variabili in C++

Come dichiarare e usare le variabili in C++.

Cos'è una variabile?

Un programma deve ricordare delle informazioni mentre lavora: il nome dell'utente, il punteggio di un gioco, il risultato di un calcolo.

Le **variabili** sono i contenitori in cui il programma memorizza queste informazioni. Pensa a una variabile come a una scatola con un'etichetta: l'etichetta è il nome della variabile, il contenuto della scatola è il suo valore.

Dichiarare una variabile

In C++ devi **dichiarare** una variabile prima di usarla. La dichiarazione specifica due cose: il **tipo** (che genere di valore conterrà) e il **nome** (come si chiama):

```
int eta;           // una scatola per numeri interi, chiamata "eta"
double altezza;    // una scatola per numeri decimali, chiamata "altezza"
char iniziale;     // una scatola per un singolo carattere, chiamata
"iniziale"
```

Dopo la dichiarazione, la scatola esiste in memoria, ma è vuota (o meglio: contiene un valore casuale imprevedibile).

Inizializzare una variabile

Inizializzare significa mettere subito un valore nella scatola al momento in cui la crei:

```
int eta = 16;
double altezza = 1.65;
char iniziale = 'A';
bool maggiorenne = false;
```

In C++ moderno puoi anche usare le parentesi graffe per inizializzare:

```
int x{10};           // equivalente a int x = 10;
double pi{3.14};
```

Questa forma è preferita dai programmatori più esperti perché il compilatore la controlla più attentamente.

Cambiare il valore

Puoi cambiare il contenuto di una variabile in qualsiasi momento con l'operatore `=` :

```
int x = 5;
cout << x;  // stampa 5

x = 10;      // cambiamo il valore
cout << x;  // stampa 10

x = x + 3;   // usiamo il valore corrente per calcolarne uno nuovo
cout << x;  // stampa 13
```

Dichiarare più variabili insieme

Puoi dichiarare più variabili dello stesso tipo in una riga sola:

```
int a, b, c;           // tre variabili intere non inizializzate
int x = 1, y = 2, z = 3; // tre variabili con valori diversi
```

Le regole per i nomi

I nomi delle variabili devono seguire queste regole:

- Devono iniziare con una lettera (`a-z` , `A-Z`) o un underscore `_`
- Possono contenere lettere, numeri e underscore
- Non possono contenere spazi o simboli speciali
- Non possono essere parole riservate del linguaggio (`int` , `if` , `return` , ecc.)
- Le maiuscole contano: `eta` e `Eta` sono variabili diverse

```
// Nomi validi
int eta;
double altezza_media;
string nomeUtente;
int x1;

// Nomi non validi
int 2x;           // inizia con un numero - ERRORE
int nome utente; // contiene uno spazio - ERRORE
int int;          // è una parola riservata - ERRORE
```

Le convenzioni di stile

Non ci sono regole fisse sullo stile, ma due convenzioni sono molto diffuse:

- **camelCase:** `nomeUtente` , `etaMinima` , `numeroDiStudenti`
- **snake_case:** `nome_utente` , `eta_minima` , `numero_di_studenti`

Scegli una e usala in modo coerente in tutto il progetto.

Attenzione: variabili non inizializzate

In C++, una variabile dichiarata ma non inizializzata contiene un valore casuale (quello che c'era in quella zona di memoria prima):

```
int x;  
cout << x; // stampa un numero imprevedibile – comportamento non definito!
```

In Python questo causa un errore chiaro. In C++ il compilatore potrebbe non avvertire, e il programma si comporta in modo strano.

Regola pratica: inizializza sempre le variabili quando le dichiari.

Esempio completo

```
#include <iostream>  
#include <string>  
using namespace std;  
  
int main() {  
    // Dichiariamo e inizializziamo quattro variabili di tipo diverso  
    string nome = "Alice";  
    int eta = 16;  
    double altezza = 1.65;  
    bool haPatente = false;  
  
    // Stampiamo i valori  
    cout << "Nome: " << nome << endl;  
    cout << "Età: " << eta << endl;  
    cout << "Altezza: " << altezza << " m" << endl;  
    cout << "Ha la patente: " << boolalpha << haPatente << endl;  
  
    // Modifichiamo il valore di eta  
    eta = 17;  
    cout << "Età aggiornata: " << eta << endl;  
  
    return 0;  
}
```

Output:

Nome: Alice
Età: 16
Altezza: 1.65 m
Ha la patente: false
Età aggiornata: 17